Queues

🔥 **Hot Tips**

- Default to the built-in lite backend for local, lightweight queuing — no external services needed
- For production, configure RabbitMQ or Kafka backends
- Install backends separately: `gem install bunny` for RabbitMQ, `gem install ruby-kafka` for Kafka
- Messages automatically retry on failure (configurable `max_retries`, default 3)
- Failed messages go to a dead-letter queue after exhausting retries

Basic Usage

Produce and consume messages with the built-in lite backend (file-based, zero config):

ruby

```ruby
require "tina4"

# Produce a message
producer = Tina4::Producer.new
producer.publish("emails", { to: "alice@example.com", subject: "Welcome" })

# Consume messages
consumer = Tina4::Consumer.new(topic: "emails")
consumer.on_message do |msg|
  puts "Processing: #{msg.payload}"
end
consumer.start
```

Backends

Lite Backend (Default)

Zero-dependency, file-based queue. Perfect for development and single-server deployments.

```ruby
backend = Tina4::QueueBackends::LiteBackend.new
producer = Tina4::Producer.new(backend: backend)
```

RabbitMQ

Requires the `bunny` gem:

```ruby
backend = Tina4::QueueBackends::RabbitMQBackend.new(
  host: "localhost",
  port: 5672,
  username: "guest",
  password: "guest"
)

producer = Tina4::Producer.new(backend: backend)
consumer = Tina4::Consumer.new(topic: "emails", backend: backend)
```

Kafka

Requires the `ruby-kafka` gem:

```ruby
backend = Tina4::QueueBackends::KafkaBackend.new(
  brokers: ["localhost:9092"],
  group_id: "my-group"
)
```

```ruby
producer = Tina4::Producer.new(backend: backend)
consumer = Tina4::Consumer.new(topic: "emails", backend: backend)
```

Producing Messages

Single message

```ruby
producer = Tina4::Producer.new
message = producer.publish("notifications", { user_id: 42, type: "alert" })
puts message.id      # UUID
puts message.topic   # "notifications"
```

Batch publish

```ruby
payloads = [
  { to: "alice@example.com", subject: "Hello" },
  { to: "bob@example.com", subject: "Welcome" }
]

messages = producer.publish_batch("emails", payloads)
```

Consuming Messages

Continuous polling

```ruby
consumer = Tina4::Consumer.new(topic: "emails", max_retries: 3)

consumer.on_message do |msg|
  puts "ID: #{msg.id}"
  puts "Topic: #{msg.topic}"
```

```ruby
  puts "Payload: #{msg.payload}"
  puts "Attempts: #{msg.attempts}"
end

consumer.start(poll_interval: 1)  # Polls every second
```

Single message pull

```ruby
consumer = Tina4::Consumer.new(topic: "emails")

consumer.on_message do |msg|
  puts msg.payload
end

consumer.process_one  # Process one message, non-blocking
```

Stopping a consumer

```ruby
consumer.stop
```

Error Handling and Retries

Messages that raise exceptions during processing are automatically retried:

```ruby
consumer = Tina4::Consumer.new(topic: "emails", max_retries: 3)

consumer.on_message do |msg|
  # If this raises, the message is requeued (up to 3 times)
  result = send_email(msg.payload)
  raise "Send failed" unless result
end

consumer.start
```

After `max_retries` attempts, failed messages are moved to a dead-letter queue.

Message Object

Each message exposes:

Method	Description
`msg.id`	Unique UUID
`msg.topic`	Topic name
`msg.payload`	The message data (Hash, String, etc.)
`msg.created_at`	Timestamp
`msg.attempts`	Number of processing attempts
`msg.status`	`:pending` , `:processing` , `:completed` , `:failed`

Integration with Routes

ruby

```ruby
Tina4.post "/api/notify", auth: false do |request, response|
  producer = Tina4::Producer.new
  message = producer.publish("notifications", request.json_body)
  response.json({ queued: true, message_id: message.id })
end
```

Further Reading

- Basic Routing — route integration
- ORM — using ORM objects as message payloads

- REST API — REST endpoints for queue management

Previous page
GraphQL